Naturally, it also works for trivial little cases that you wouldn't really want to do unless you were trying to learn how to compose:

```
addFive = (x) -> x + 5
multiplyByThree = (x) -> x * 3
multiplyByThreeAndThenAddFive = compose addFive, multiplyByThree
multiplyByThreeAndThenAddFive 10
# 35
```

The function on the right (the second argument to compose) is evaluated first.

Remember that functions are a bit like pipes. You can explicitly take output from one pipe and use it as the input to another pipe, or you can just connect the pipes together. `Compose` isn't the only way you might want to connect functions together. In figure 6.2 you can see `compose`, as well as the other basic combinators `before`, `after`, and `around` that will be discussed in this section.

Figure 6.2. Visualizing `compose`, `before`, `after`, and `around`